ROLL DRAUF · ENGINEERING

Creation Engine v1.0

Adversarial audit

> Historical findings · Reviewed on 23 September 2026

23 September 2026
Theoretical design and review package

No application implementation, live migration or production-readiness claim. Companion contracts and acceptance scenarios are included in the review archive.

Historical audit — superseded design reviewed in v1.1; findings below describe v1.0.

23 September 2026 · Review of v1.0, dated 22 September 2026 · Discover / read-only review

Verdict: retain the product direction; revise the specification before committing to production integration. The document makes useful separations between definitions and instances, activities and effects, authorship and interpretation, and mechanics and presentation. It does not yet define enough behavior for two independent implementers to build compatible engines. Its largest risks are runtime correctness, compatibility with the existing VTT, and preservation of secrets through derived views.

The intended scope is explicitly the full target system. This review does not penalize that ambition or propose silently shrinking it. It distinguishes a target vision from the contracts and evidence needed to implement it safely.

Scope and evidence. I read all 41 PDF pages, including appendices; the channel discussion described it as 42 pages. The source is the IT Keller attachment message, posted on 22 September at 15:17 UTC. The local PDF (`Roll_Drauf_Creation_Engine_Research_and_Technical_Specification.pdf`; retained with the original local audit), extracted text (`creation-engine-source.txt`; retained with the original local audit), and source manifest (`source-manifest.json`; retained with the original local audit) preserve the reviewed version. All page references below refer to the PDF's printed page numbers.

VTT evidence comes from `/home/admin/projects/roll-drauf-vtt`, HEAD `94ff1b9e9c07e048151469195f262fda1d80536b`, including its pre-existing uncommitted changes. File hashes are in the manifest. This is static source inspection, not a claim that this exact tree is deployed. No application code, database, deployment, or Discord messages were changed. The scenarios below are proposed acceptance tests and reasoning counterexamples; they were not executed against a Creation Engine implementation.

High = could corrupt state, expose restricted information, or force substantial redesign if left unresolved in the affected release. Medium = materially weakens delivery confidence, portability, or acceptance criteria. These are primarily specification findings, not claims of demonstrated production exploits.

ID	Severity	Finding	Primary document evidence
F01	High	Existing VTT integration and migration have no ownership contract	§§18, 43, 55–56, 85; pp. 10, 20, 24, 33
F02	High	Runtime commands lack atomic commit and durable retry semantics	§§56, 63–64, 71, 76; pp. 24, 26–29
F03	High	Human resolution and reaction windows have no resumable lifecycle	§§30, 55–57; pp. 15, 24
F04	High	Effects, time, and formulas are named but not operationally specified	§§27–33; pp. 14–16
F05	High	Definition pinning does not freeze the full execution environment	§§60, 65–70; pp. 25, 27–28
F06	High	Field secrecy does not yet cover every derived/read/write path	§§34, 47, 72–77; pp. 16, 21, 29–30

ID	Severity	Finding	Primary document evidence
F07	High	Declarative content still needs bounded execution and import rules	§§33, 69, 75, 83; pp. 16, 28–29, 32
F08	High	Reusable definitions and campaign-world identity are conflated	§§22–24, 48–52, 65; pp. 12–13, 22–23, 27
F09	High	Provenance alone does not make progression or runtime undo reversible	§§32, 41, 45, 57; pp. 16, 19–20, 24
F10	High	Guided compilation lacks a bounded language and question-flow contract	§§35–42, 81, 85; pp. 17–19, 32–33
F11	Medium	Ruleset independence conflicts with closed D&D-shaped primitives	§§17/P9, 28–29, 87; pp. 10, 14–15, 34
F12	Medium	Delivery sequence postpones its strongest falsification test	§§79–86; pp. 31–34
F13	Medium	Evidence confidence and normative requirements need reconciliation	§§4, 7, 37, 42; Appendix B; pp. 3–6, 17, 19, 38–39

F01 — Specify the integration boundary before adding another authoritative state model.

The target character model is a projection of choices, definitions, and runtime state. The existing VTT persists `ac`, `hp_max`, ability scores, and proficiency directly in Character (`vtt/models/character.py:29`). Its update endpoint (`vtt/characters/routes.py:398`) accepts direct changes to many of those fields. Equipment and inventory are separate per-character tables, without the proposed definition/revision references in their inspected models. Existing progression uses Roll Drauf Light content snapshots and levels 1–5 in progression.py (`vtt/roll_drauf/progression.py:19`); a level-up also changes an ORM ability score (`vtt/characters/routes.py:540`).

There is already cross-surface runtime state: TokenState (`vtt/models/token_state.py:19`) stores HP and references a character; combat HP adjustment (`vtt/combat/service.py:273`) updates both. An editor cannot safely become authoritative without specifying how these writers cooperate.

Counterexample: a legacy sheet edit sets AC to 18 while an engine projection calculates 19; a token or export then reads the other representation. Alternatively, an old manually entered modifier is imported as a base value and applied a second time as an effect.

Required closure: an integration inventory assigning authority for each migrated field, legacy write behavior, ID mapping, imports/exports, token synchronization, and manual overrides. Define how unknown historical provenance is represented; it cannot be reconstructed reliably from a finished total. Include migration parity and rollback requirements. Resolve whether the first engine ruleset serves existing Roll Drauf Light characters, D&D characters, or both. The repository's domain context (`CONTEXT.md:1`) also distinguishes local accounts, campaign roles, and optional Discord identity: `WorkspaceId` must not silently become a Discord guild ID.

Pass condition: a migrated character retains its intentional values and overrides; old clients cannot create a competing mechanical truth; sheet, token, invocation, and export agree on state and revision.

F02 — Make action resolution atomic and replayable across retries and restarts.

The document includes an idempotency key, optimistic revisions, audit events, and concurrency tests. These are good requirements, but their joint semantics are absent. The resolution pipeline emits results before applying effects and recording audit; it does not identify an authoritative commit point or say that early results are provisional. `expectedRevision` covers one draft/instance, whereas an action can consume a party resource and affect several actors.

Counterexample: two players spend the final shared charge concurrently. Or the server spends a charge and rolls damage, crashes before responding, and receives the same invocation after restart. A key in the request alone does not decide whether to return the old roll, reroll, reject, or spend again.

The current VTT has durable combat events (`vtt/models/combat_event.py:8`), but the inspected socket deduplication cache (`vtt/socket_handlers.py:338`) and broadcast sequence counters (`vtt/utils/realtime.py:8`) are process-local. These helpers are not evidence of a durable command protocol.

Required closure: specify the action's atomic write set; authorization at commit; cross-entity conflict handling; idempotency scope, retention, and request-payload binding; persisted outcome and random-roll evidence; crash recovery; and event publication relative to commit. Distinguish security audit, domain events, and client notifications. Replays must not reroll or respend.

Pass condition: race two spends with one charge, then inject failures before and after commit and retry after restart. Exactly one action consumes it; retries of that action return the same committed outcome; clients can recover missed notifications.

F03 — Human predicates require durable pending actions, not just a prompt field.

`ASK_PLAYER`, `ASK_GM`, and manual fallback are central to the product. The model has no action-scoped prompt identity, responder authority, expiry, cancellation, stale-context handling, or durable resume contract. The straight-line runtime pipeline also does not define interrupt/reaction windows or replacement effects around rolls and damage.

Counterexample: a player is asked to empower a hit. While the dialog is open, the target moves, another action spends the charge, the player loses campaign access, or the server restarts. The old answer arrives twice. Which hit and which state does it authorize?

Required closure: define pending-action states and the information that binds a response to its action, target, state version, authorized responder, and decision window. Define whether relevant state changes invalidate or re-evaluate the proposal. Specify timeouts, GM intervention, reaction ordering, and downstream-trigger causality. Avoid holding a database transaction open while waiting for a person.

Pass condition: all of the above interleavings produce one documented outcome without spending twice, applying to a different target, or accepting an unauthorized answer.

F04 — Determinism requires an effect algebra and a clock contract.

`operator`, `priority`, and `stackingPolicy` are placeholders, not a complete evaluator contract. For a base value of 10, applying `ADD 2` then `MULTIPLY 2` gives 24; reversing them gives 22. Both fit the current structures. Equal priority, two `SET` effects, typed collection operations, unit conversion, rounding, absent values, and dependency reads of base versus modified values remain undefined. A dependency graph cannot by itself resolve these semantics.

Duration and cadence are correctly separated, but `ONCE_PER_TURN` lacks a defined reset identity: every combatant's turn, the source's turn, or some other ruleset boundary. Reordering initiative, rewinding combat, leaving combat, and game-world dawn versus server clock time need explicit behavior. A formula graph can be acyclic while event triggers still recurse indefinitely.

Required closure: define typed operators and their ordering; tie-breaking or conflict rejection; modifier layers; stacking/source identity; numeric and error behavior; time-domain identifiers; and trigger scheduling rules. Fixed-point evaluation needs convergence and failure semantics if supported.

Pass condition: reorder equivalent serialized input and evaluate on supported execution paths; obtain the same result and explanation. Test duplicate sources, competing setters, rounding boundaries, initiative changes, and chained triggers. Where order is intentionally meaningful, preserve and expose that order.

F05 — Pin the complete execution context, not only the visible item revision.

Immutable revisions and compatible dependency ranges do not alone preserve a campaign. A pinned definition can still resolve another dependency version or run under changed ruleset, formula, schema, or engine semantics. Renderer-compatible ranges likewise express allowed change, not exact reproduction of an old publication.

Counterexample: `flamebrand@1.3.0` remains pinned while its dependency resolves a newer fire-damage handler. An engine upgrade changes stacking order. Its content identifier is unchanged but damage changes. A migration reduces maximum charges from three to one while an instance currently has two.

Required closure: define a resolved, retained execution context covering transitive content, ruleset implementation, relevant evaluator/schema versions, and assets needed for the promised reproduction level. Specify migration of current resources, active effects, choices, and pending invocations. Define instance overrides versus upstream changes, update timing, rollback, and withdrawal/unavailability of referenced packages. Differentiate exact replay, semantic compatibility, and approximate historical presentation.

Pass condition: reinstall or restore the same campaign later and reproduce its agreed outcomes; preview an update, migrate live state under a specified policy, and roll back without silently clamping or discarding information.

F06 — Apply secrecy to projections, explanations, exports, and mutation responses.

The document explicitly covers field-level visibility, permission-filtered search, secret-safe relationships, and View As. What is missing is composition: policy inheritance/deny precedence, derived data, permission changes, audience-specific caches, and write authorization. Permission to view a definition is not permission to publish it, invoke it, edit a player resource, or use it in a campaign.

There is a concrete integration hazard. _can_read_character (`vtt/characters/routes.py:233`) permits the owner or an active campaign member; _serialize_character_sheet (`vtt/characters/routes.py:241`) returns the complete `character_data` blob. Storing the new secret components inside that blob without changing the serializer would expose them to those members. This is a conditional integration failure established by the read path, not a claim that such secrets already exist there.

Counterexample: a hidden curse modifies AC. The public value is visible, but `/explain/{path}` reveals the curse name, source ID, or private author decision. A semantic conflict diff or exported package reveals the same information. A GM-warmed cache is reused for a player.

Required closure: an authorization matrix for commands and every output surface, including WebSocket deltas, snapshots, provenance, errors, exports, simulations, PDFs, and telemetry. Define permitted disclosure of a derived value whose inputs are secret, and revocation behavior for cached/subscribed data. Imported ownership/grants and client-provided `actorId` must not establish authority. Existing audience-scoped broadcasts (`vtt/play/routes.py:67`) are useful infrastructure, but do not establish arbitrary field-policy support.

Pass condition: GM/player/owner-private and cross-campaign fixtures reveal only the authorized representation through every relevant API and transport, including after membership changes.

F07 — Data-only packages still execute work and require limits.

No arbitrary `eval`, sandboxed scripts, and quotas are useful controls. However, formulas, selectors, relationship traversals, fixed-point evaluation, and event subscriptions are executable behavior even when encoded as data. Request-rate limits do not bound one pathological request.

Counterexample: an allowed resource-change trigger modifies the same resource; two triggers activate one another; an area action expands to a huge target set; a deep acyclic formula graph exhausts evaluation capacity. A package may also reference external renderer assets or include extreme archive expansion without containing executable code.

Required closure: limits for graph size/depth, evaluation steps, trigger fan-out, targets, iterations, simulation work, memory, and runtime duration, with documented failure behavior. Bound import size and expansion; specify reference/asset URL handling and archive-path safety where those formats are supported. Define whether a limit failure occurs before commit or has an explicit recovery outcome. Capability declarations require enforcement even for trusted extensions.

Pass condition: adversarial but schema-valid packages are rejected or terminated within the chosen budgets without partially mutating a campaign or degrading unrelated sessions.

F08 — Separate reusable content identity from campaign occurrence and world state.

The definition/revision/instance split is strong, but the world model then places a named NPC, spatial presence, faction membership, and secrets under one `EntityDefinition`. Relationships only expose `sourceId` and `targetId`; they do not state whether these refer to definitions, revisions, campaign entities, or instances.

Counterexample: two campaigns install the same named NPC package. One campaign kills that NPC or changes its allegiance. A third placement is a disguise of the same campaign individual. The current structures do not decide what is shared, what is independent, and what gets a new identity. Campaign-specific facts must not accidentally become publisher revisions.

Required closure: specify reference kinds and identity scope; definition versus occurrence relationships; mutable world-state ownership; behavior of forks and imports; instance-local overrides; and deletion/tombstone behavior for stable references. Shared identity infrastructure need not imply shared mutation or permission boundaries.

Pass condition: package reuse, two independent campaign instances, and multiple representations of one campaign entity behave distinctly and predictably through edits, exports, upgrades, and deletion.

F09 — Define source multiplicity and compensation, not just provenance.

Reversible progression is promised, but grants and their lifecycle are underspecified. Provenance records identify sources; they do not specify whether two grants create one effective capability with two sources, two instances, or an illegal duplicate. Nor does tracing a mutation prove it can be reversed after other actions depend on it.

Counterexample: a species and a class both grant the same proficiency. Removing the class level must preserve the species grant. A class-granted item has been transferred or consumed. Removing the level cannot safely delete another character's current inventory or undo later combat outcomes.

Required closure: define grant identity, multiplicity, deduplication, dependent-choice invalidation, resource maxima/current-state reconciliation, and the distinction between recalculation, reversal, and compensating changes. Runtime undo needs conflict rules after subsequent actions. Multiple authorized sources must survive removal of any one source.

Pass condition: grant the same capability through two paths, remove each independently, then test a transferred/consumed grant and an undo after later state changes. Each has an explicit result or a reviewable unresolved conflict.

F10 — Bound what the Guided Compiler understands and prove the interview works.

A deterministic schema-guided interview is feasible without an LLM. That does not establish unrestricted natural-language understanding or usable question selection. The packet supplies field metadata but no accepted intent grammar/vocabulary, coverage registry, rule priority, termination guarantee, or versioned question/answer semantics. `answerAdapter`, predicates, and query expressions themselves need typed, validated contracts; the pseudostructures are not executable schemas.

The phase plan puts the natural-language importer in Phase 8, while the primary Guided example starts from free prose. Clarify the earlier entry experience. In the example, a sword bursting into flames immediately proposes a fire-damage activity. Harmless light, intimidation, or visual transformation are also possible. Because it is a proposal, this is not proof of a silent committed mutation; nevertheless, its subsequent damage question illustrates how an interview can steer a creative choice before establishing intent.

Required closure: define supported structured intents and any controlled-language input, unknown/ambiguous input behavior, explanatory proposal confirmation, deterministic question ranking, contradictory answers, and completeness criteria. Persist stable question IDs and schema/compiler versions; changing an earlier answer must invalidate affected descendants safely. Bound the promise to supported mechanics while providing explicit extension/manual paths.

Pass condition: a fixture corpus includes harmless flames, ambiguous recharge, negation, unsupported mechanics, contradictory answers, locale changes, and Guided → Advanced → Guided edits. Equivalent accepted decisions produce equivalent canonical results; no necessary question is skipped and no hidden creative choice is committed. Separately, novice task trials must show users can complete representative mechanics without learning internal API terms.

F11 — Demonstrate ruleset independence before freezing the kernel.

The platform's independence promise conflicts with closed core enums for short/long rests, dawn, initiative, turns, and rounds. `CUSTOM_EVENT` is an escape hatch, but does not specify how custom recovery/time semantics receive validation, guided questions, localization, simulation, and rendering. The ruleset SDK decision is deferred until the second game system, after the first kernel may already fix these assumptions.

Required closure: identify which concepts belong to the core and which belong to versioned ruleset contracts. Establish first-ruleset/edition mixing policy. Use a small contrasting ruleset fixture—such as a progress clock or success-pool mechanic—to test the boundary before claiming a universal engine. This does not require shipping a second full game system.

Pass condition: the contrasting fixture works through supported extension contracts without editing core business logic or falling back to opaque custom blobs.

F12 — Move evidence of integration and usability earlier in the delivery decision.

Phase 1 builds almost the entire rules vocabulary, including progression primitives, before the first integrated slice in Phase 2. That slice must already exercise three editors, several outputs, runtime, package updates, and provenance. Calling its primitives final-system primitives cannot establish their finality before those paths have been tested.

The testing inventory is broad and useful. Missing are measurable workloads/latencies, explicit nonfunctional budgets, milestone owners/dependencies, and go/no-go evidence. Incremental recomputation alone does not bound work: one edit may legitimately affect most of an actor or a large graph. A magic-weapon demonstration also cannot prove class progression or campaign-world identity.

Required closure: attach acceptance evidence and implementation assumptions to each milestone; define benchmark workloads and budgets; identify which unresolved contracts block the first slice versus later capabilities. Require early proof of a real existing-character integration and the guided interaction. Preserve the full target scope while allowing abstractions to be revised based on those results.

Pass condition: milestone completion is demonstrated by reproducible outcomes, a bounded workload, and a stated compatibility boundary—not merely by the existence of each subsystem.

F13 — Tighten evidence classification and requirement consistency.

The central competitor references checked are real and relevant:

Claim checked	Result and primary source
Foundry separates item activities and supports extensible activity types	Supported by Activities.
Foundry D&D5e 6.0 area behaviors include active effects and difficult terrain	Supported by Activity Behaviors.
Dungeon Master's Vault derives entities from choices/ modifiers and orders dependencies	Supported by the project's architecture description.
Roll20 offers custom character options, future-level planning, and contextual modifier queries	Supported by its Homebrew Character Builder announcement.
D&D Beyond describes composable data and a cloud ECS approach	Supported by its 18 August 2026 update. The same update describes early internal testing; it does not establish a mature public homebrew platform.
Fantasy Grounds separates effect lifetime and expenditure	Supported by its advanced effects documentation.
Kanka supports member-view testing with an owner-private exception	Supported by Testing permissions.
Homebrewery has undertaken renderer sanitization work	Supported by its release history; that does not establish the safety of a proposed Roll Drauf renderer.

This was a targeted primary-source check, not an exhaustive revalidation of all 35 register entries. Community anecdotes were not used here to establish implementation facts.

The document's own evidence policy distinguishes official technical evidence from community friction. Yet §7 labels DiceCloud evidence A/B while citing a community technical guide. That guide may be accurate, but the classification and attribution need reconciliation or a primary implementation citation. Public architecture descriptions demonstrate a pattern's existence; they do not prove that combining the patterns achieves Roll Drauf's usability, performance, or delivery goals. Those are hypotheses needing local evidence.

There are also normative inconsistencies: §37 makes schema-generated question flow MUST, while CE-GUIDE-003 says SHOULD; §32 qualifies reversibility while CE-CHAR-002 states it more strongly; body text and catalogue differ in strength on human fallback. §42 combines mechanical completeness and publication eligibility into a ladder without stating whether a lore-only package can be published. Renderer ranges also need to be distinguished from exact artifact stability.

Required closure: a single requirements catalogue with precedence, scope, test IDs, and traceability; explicit confidence per competitor claim; and an eligibility matrix separating publication permissions from automation completeness.

Acceptance scenarios to attach to the revised plan. These are proposed tests, not reported results.

Scenario	Required invariant	Findings
Existing character migrated; legacy client edits AC	One authoritative result; intentional overrides preserved	F01
Two invocations spend the last party charge	At most one succeeds; no negative resource	F02
Crash after commit, retry after restart	Same outcome; no new roll/spend/ effect	F02
Prompt answered after movement, resource change, access revocation	Response rejected or re-evaluated under documented rules	F03
Same effects serialized in different orders	Same result unless order is explicitly semantic	F04
Initiative changes and cadence resets	No extra or skipped use beyond the declared rule	F04
Pinned campaign restored after engine/package changes	Declared reproduction guarantee holds	F05
Public number depends on a hidden effect	Permitted value disclosure; private explanation stays private	F06
GM warms cache; player requests same object/export	No audience or campaign leakage	F06
Schema-valid trigger loop or excessive simulation	Bounded termination; no partial campaign mutation	F07
Same NPC package installed in two campaigns	One campaign's events do not rewrite the other	F08
Same proficiency granted twice; one grant removed	Remaining grant still applies	F09
Consumed/transferred item followed by progression rollback	Explicit compensation/conflict; no blind deletion	F09
Harmless flaming sword and contradictory interview answers	No damage invented; contradictions remain visible	F10

Scenario	Required invariant	Findings
Small non-D&D mechanic through the ruleset boundary	No core fork or opaque bypass	F11

What should survive the revision. Keep shared validated commands across authoring surfaces; immutable published definitions; explicit ambiguity; manual/human fallback; separation of duration from cadence; provenance; and permission-aware presentation. These are useful constraints. “Compiler” should mean a specified transformation with diagnostics and reproducible behavior, not a substitute for defining the semantics being compiled.

Next reviewable step. Produce a bounded v1.1 contract addendum covering F01–F10, with named decisions and executable examples. Before production schema lock or implementation commitments, resolve: first supported ruleset and legacy migration boundary; authoritative action/commit/retry behavior; effect/time semantics; exact version-retention guarantees; and policy propagation to every output. The evaluation mechanisms and pass conditions above are review requirements, not an approved implementation architecture. F11–F13 should shape the extension boundary, milestone gates, and requirements register.

Audit delivery is complete; implementation readiness remains conditional on that rework. The repository's Discover workflow was used for evidence and risk inventory. No transition into implementation was attempted. Review artifacts and the source PDF are retained locally as task evidence; the reduced channel context is discovery evidence and is not intended for publication.